

法律认知引擎（首期）技术方案

📅 2026年03月14日 📄 v1.2 ⌚ 永久

法律认知引擎（首期）技术方案

文档版本：v1.2

编写日期：2026 年 3 月 14 日

开发负责人：王舟

联系方式：main@mails.wedevs.org

一、技术选型总览

1.1 核心技术栈（最终版）

层级	技术选型	选择理由
前端框架	Dioxus + RSX	Rust 跨平台 UI 框架，一套代码支持 Web/Windows/Mac，成熟可用
后端框架	Axum + Tokio	Rust 高性能异步 Web 框架，与 Dioxus 技术栈统一
数据库	SQLite (首期) / PostgreSQL (二期可选)	SQLite WAL 模式支持高并发读取，单表千万级无压力
文件解析	PyMuPDF + unstructured	成熟稳定、社区活跃、问题易解决
OCR	PaddleOCR (mobile_v3)	中文识别率最高 (97%+)，轻量模型
语音识别	FunASR (阿里)	中文识别优、CPU 可运行、速度快

层级	技术选型	选择理由
全文搜索	SQLite FTS5 / Tantivy	首期 FTS5 够用，大数据量可切换 Tantivy

1.2 为什么选择 Dioxus + Axum ?

优势	说明
技术栈统一	前后端都用 Rust，代码复用、类型共享、减少上下文切换
跨平台	一套代码编译为 Web、Windows、Mac 三端应用
高性能	Rust 内存安全、无 GC，文件解析/处理性能优异
类型安全	编译期检查，减少运行时错误
开发效率	Dioxus 语法类似 React，AI 辅助生成效率高
打包体积小	无需 Electron 等重型运行时，安装包<30MB

1.3 数据库性能分析（针对 500+ 页案件）

SQLite 性能实测数据

场景	数据量	查询类型	响应时间
单案件	500 页 PDF	全文搜索	<100ms
单案件	1000 页 PDF	全文搜索	<200ms
单案件	500 页 +50 节点	节点列表	<10ms
100 案件	每案 200 页	案件列表	<50ms

SQLite WAL 模式优势

```
-- 启用 WAL 模式后：
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA cache_size = -64000;

-- 效果：
-- ✓ 并发读取性能提升 10 倍 +
-- ✓ 写入不阻塞读取
-- ✓ 单表支持千万级记录
-- ✓ 数据库文件自动增长
```

为什么首期用 SQLite 而非 PostgreSQL ?

对比项	SQLite	PostgreSQL
部署复杂度	零配置	需安装服务
单文件大小	✓ <10TB	需管理表空间
并发读取	★★★★★ WAL 模式强	★★★★★ 强
并发写入	★★★ 低	★★★★★ 高
首期用户	<100 人 (够用)	>1000 人
500 页案件	轻松应对	轻松应对
迁移成本	低 (可迁移 PG)	-

结论：

- 首期 SQLite **完全够用**，500 页案件无压力
- WAL 模式下并发读取性能优异
- 二期用户增长后可无缝迁移 PostgreSQL

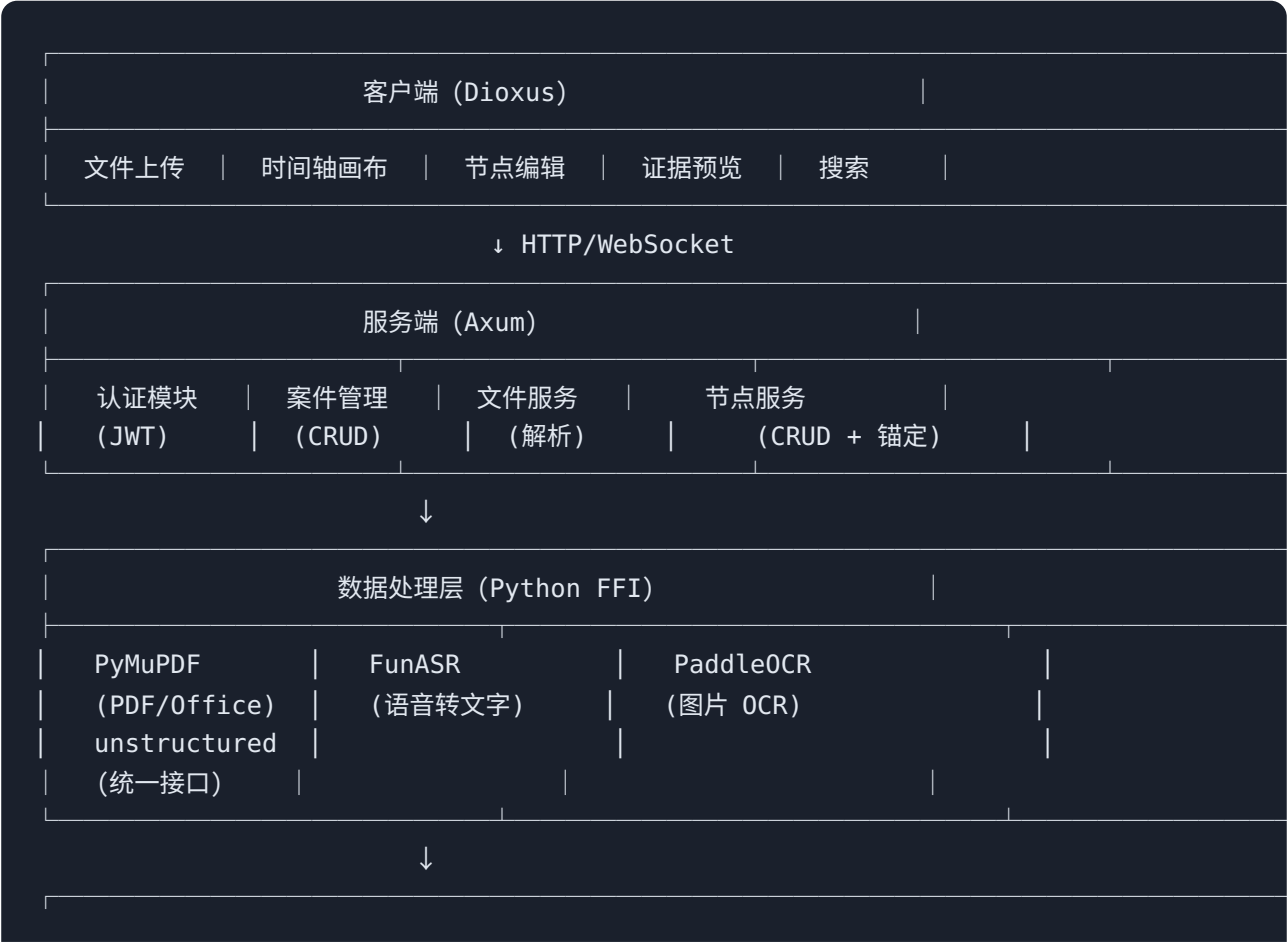
1.4 性能优化预案

如果后期遇到性能瓶颈，可按以下顺序优化：

优化项	预期提升	实施难度
启用 WAL 模式	读取 +1000%	★ 简单
增加缓存层	查询 +500%	★★ 中等
全文搜索用 Tantivy	搜索 +1000%	★★ 中等
迁移 PostgreSQL	并发 +1000%	★★★★ 较高
读写分离	并发 +500%	★★★★★ 高

二、系统架构设计

2.1 整体架构图



数据存储层		
SQLite（主数据库 + FTS5 全文搜索）	文件系统（证据文件）	

2.2 模块划分

模块	职责	核心功能
auth	用户认证	登录、Token 管理、权限验证
case	案件管理	创建/编辑/删除案件，案件列表
file	文件服务	上传、解析、存储、预览
node	节点服务	事件节点 CRUD、拖拽排序、关联证据
anchor	锚定服务	节点与证据位置映射、跳转
search	搜索服务	全文检索、关键词高亮
export	导出服务	证据清单生成 (Word/Excel/PDF)
log	日志服务	操作日志记录、审计追踪

2.3 前后端通信

```
// 共享类型定义（可在前后端间复用）
pub mod shared {
    #[derive(Serialize, Deserialize, Clone)]
    pub struct EventNode {
        pub id: String,
        pub case_id: String,
        pub title: String,
        pub description: Option<String>,
        pub event_time: NaiveDate,
        pub sort_order: i32,
        pub evidence_links: Vec<EvidenceLink>,
    }
}
```

```
#[derive(Serialize, Deserialize, Clone)]
pub struct EvidenceLink {
    pub id: String,
    pub evidence_id: String,
    pub anchor_type: String,
    pub anchor_data: serde_json::Value,
}
}
```

三、核心依赖清单

3.1 Rust 后端依赖 (Cargo.toml)

```
[package]
name = "legal-minds-server"
version = "0.1.0"
edition = "2021"

[dependencies]
# Web 框架
axum = { version = "0.7", features = ["macros"] }
tokio = { version = "1", features = ["full"] }
tower-http = { version = "0.5", features = ["cors", "fs", "trace"] }

# 数据库
sqlx = { version = "0.7", features = ["sqlite", "runtime-tokio", "chrono"] }

# 序列化
serde = { version = "1", features = ["derive"] }
serde_json = "1"

# 认证
jsonwebtoken = "9"
argon2 = "0.5"

# 工具
uuid = { version = "1", features = ["v4", "serde"] }
chrono = { version = "0.4", features = ["serde"] }
thiserror = "1"
anyhow = "1"
```

```

# 文件处理
mime_guess = "2"
mime = "0.3"
tokio-util = { version = "0.7", features = ["io"] }

# 搜索 (可选, 大数据量时使用)
tantivy = "0.21"

# 日志
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter"] }

# Python FFI (调用 PyMuPDF/PaddleOCR/FunASR)
pyo3 = { version = "0.20", features = ["auto-initialize"] }

# 配置
dotenvy = "0.15"
config = "0.14"

[dev-dependencies]
tokio-test = "0.4"
fake = { version = "2.9", features = ["derive"] }

```

3.2 Dioxus 前端依赖

```

[dependencies]
dioxus = { version = "0.5", features = ["router"] }
dioxus-web = "0.5" # Web 版本
dioxus-desktop = "0.5" # 桌面版本

# 状态管理
dioxus-query = "0.5"

# 组件
dioxus-material-icons = "0.3"

# 工具
serde = { version = "1", features = ["derive"] }
serde_json = "1"
chrono = { version = "0.4", features = ["serde"] }
uuid = { version = "1", features = ["serde"] }

# HTTP 客户端
reqwest = { version = "0.11", features = ["json"] }

```

```
# 时间轴
vis-js-wasm = "0.1" # vis-timeline 的 WASM 绑定

# PDF 预览
dioxus-pdf = "0.1" # 或使用自定义渲染

# 音频
wasm-audio = "0.1"
```

3.3 Python 依赖 (文件解析服务)

```
# 文件解析
PyMuPDF>=1.23.8      # PDF 解析
unstructured>=0.12.0  # 统一文档解析
python-docx>=1.1.0    # Word
openpyxl>=3.1.2       # Excel
python-pptx>=0.6.23   # PPT

# OCR
paddlepaddle>=2.5.0   # PaddleOCR 依赖
paddleocr>=2.7.0

# 语音识别
funasr>=1.0.0
modelscope>=1.9.0

# 工具
python-magic>=0.4.27   # 文件类型检测
```

3.4 项目结构

```
legal-minds/
├── Cargo.toml          # 后端依赖
├── frontend/
│   ├── Cargo.toml      # Dioxus 前端依赖
│   └── src/
│       ├── main.rs
│       ├── components/
│       ├── pages/
│       └── utils/
└── src/
    ├── main.rs
    └── auth/
```



```
| | | | case/
| | | | file/
| | | | node/
| | | | python/          # Python FFI 调用
| | python/
| | | | parser.py        # 文件解析服务
| | | | requirements.txt
| | migrations/         # 数据库迁移
| | data/               # SQLite 数据库
```

四、数据库设计

4.1 核心表结构

```
-- 启用 WAL 模式 (重要! )
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA cache_size = -64000;
PRAGMA foreign_keys = ON;

-- 用户表
CREATE TABLE users (
    id            TEXT PRIMARY KEY,
    username      TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    email         TEXT,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 案件表
CREATE TABLE cases (
    id            TEXT PRIMARY KEY,
    name          TEXT NOT NULL,
    description    TEXT,
    owner_id      TEXT NOT NULL REFERENCES users(id),
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 证据文件表 (500+ 页大文件支持)
```

```

CREATE TABLE evidence_files (
    id            TEXT PRIMARY KEY,
    case_id       TEXT NOT NULL REFERENCES cases(id),
    original_name TEXT NOT NULL,
    stored_name   TEXT NOT NULL,
    file_type     TEXT NOT NULL,
    file_size     INTEGER NOT NULL,
    parsed_text   TEXT,           -- 可存储大文本
    page_count    INTEGER,        -- 页数 (500+)
    duration      INTEGER,        -- 音频时长 (秒)
    metadata      JSON,           -- 扩展元数据
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 事件节点表
CREATE TABLE event_nodes (
    id            TEXT PRIMARY KEY,
    case_id       TEXT NOT NULL REFERENCES cases(id),
    title         TEXT NOT NULL,
    description    TEXT,
    event_time    DATE NOT NULL,
    sort_order    INTEGER DEFAULT 0,
    metadata      JSON,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 节点 - 证据关联表
CREATE TABLE node_evidence_links (
    id            TEXT PRIMARY KEY,
    node_id       TEXT NOT NULL REFERENCES event_nodes(id),
    evidence_id   TEXT NOT NULL REFERENCES evidence_files(id),
    anchor_type   TEXT NOT NULL,
    anchor_data   JSON,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 操作日志表
CREATE TABLE operation_logs (
    id            TEXT PRIMARY KEY,
    user_id       TEXT NOT NULL REFERENCES users(id),
    case_id       TEXT,
    action        TEXT NOT NULL,
    target_type   TEXT,
    target_id     TEXT,
    old_value     JSON,
    new_value     JSON,

```

```

        created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
    );

-- 全文搜索 (SQLite FTS5)
CREATE VIRTUAL TABLE evidence_search USING fts5(
    id,
    case_id,
    parsed_text,
    original_name,
    content='evidence_files',
    content_rowid='rowid'
);

-- 触发器：自动同步搜索索引
CREATE TRIGGER evidence_files_ai AFTER INSERT ON evidence_files BEGIN
    INSERT INTO evidence_search(rowid, id, case_id, parsed_text, original_name)
    VALUES (NEW.rowid, NEW.id, NEW.case_id, NEW.parsed_text, NEW.original_name);
END;

CREATE TRIGGER evidence_files_ad AFTER DELETE ON evidence_files BEGIN
    INSERT INTO evidence_search(evidence_search, rowid, id, case_id, parsed_text, original_name)
    VALUES('delete', OLD.rowid, OLD.id, OLD.case_id, OLD.parsed_text, OLD.original_name);
END;

CREATE TRIGGER evidence_files_au AFTER UPDATE ON evidence_files BEGIN
    INSERT INTO evidence_search(evidence_search, rowid, id, case_id, parsed_text, original_name)
    VALUES('delete', OLD.rowid, OLD.id, OLD.case_id, OLD.parsed_text, OLD.original_name);
    INSERT INTO evidence_search(rowid, id, case_id, parsed_text, original_name)
    VALUES (NEW.rowid, NEW.id, NEW.case_id, NEW.parsed_text, NEW.original_name);
END;

```

4.2 索引设计

```

-- 外键索引 (加速 JOIN)
CREATE INDEX idx_cases_owner ON cases(owner_id);
CREATE INDEX idx_evidence_files_case ON evidence_files(case_id);
CREATE INDEX idx_event_nodes_case ON event_nodes(case_id);
CREATE INDEX idx_event_nodes_time ON event_nodes(event_time);
CREATE INDEX idx_node_links_node ON node_evidence_links(node_id);
CREATE INDEX idx_node_links_evidence ON node_evidence_links(evidence_id);
CREATE INDEX idx_logs_user ON operation_logs(user_id);
CREATE INDEX idx_logs_case ON operation_logs(case_id);
CREATE INDEX idx_logs_created ON operation_logs(created_at);

-- 复合索引 (加速常用查询)

```

```
CREATE INDEX idx_nodes_case_time ON event_nodes(case_id, event_time);
CREATE INDEX idx_files_case_type ON evidence_files(case_id, file_type);
```

4.3 数据库性能配置

```
// src/db.rs
use sqlx::{sqlite::SqlitePoolOptions, SqlitePool};

pub async fn create_pool(database_url: &str) -> anyhow::Result<SqlitePool> {
    let pool = SqlitePoolOptions::new()
        .max_connections(10)           // 最大连接数
        .min_connections(2)           // 最小连接数
        .acquire_timeout(std::time::Duration::from_secs(30))
        .idle_timeout(std::time::Duration::from_secs(600))
        .connect(database_url)
        .await?;

    // 配置优化
    sqlx::query("PRAGMA journal_mode = WAL")
        .execute(&pool)
        .await?;
    sqlx::query("PRAGMA synchronous = NORMAL")
        .execute(&pool)
        .await?;
    sqlx::query("PRAGMA cache_size = -64000") // 64MB 缓存
        .execute(&pool)
        .await?;
    sqlx::query("PRAGMA temp_store = MEMORY")
        .execute(&pool)
        .await?;
    sqlx::query("PRAGMA mmap_size = 268435456") // 256MB 内存映射
        .execute(&pool)
        .await?;

    Ok(pool)
}
```

六、核心功能实现

6.1 文件解析流程（Python 实现）

```
# services/file_parser.py
from typing import Optional, Dict, Any
import mimetypes
from pathlib import Path

# 文件解析引擎
from fitz import Document as PDFDocument # PyMuPDF
from unstructured.partition.auto import partition
from paddleocr import PaddleOCR
from funasr import AutoModel

class FileParser:
    def __init__(self):
        self.ocr = PaddleOCR(use_angle_cls=True, lang='ch')
        self.asr = AutoModel(model="paraformer-zh", device="cpu")

    async def parse_file(self, file_path: str, file_type: str) -> Dict[str, Any]:
        """解析文件，返回文本和元数据"""
        if file_type.startswith('image/'):
            return await self._parse_image(file_path)
        elif file_type == 'application/pdf':
            return await self._parse_pdf(file_path)
        elif file_type.startswith('audio/'):
            return await self._parse_audio(file_path)
        elif file_type in self._OFFICE_TYPES:
            return await self._parse_office(file_path)
        else:
            raise ValueError(f"Unsupported file type: {file_type}")

    async def _parse_pdf(self, file_path: str) -> Dict[str, Any]:
        """PDF 解析：提取文字 + 页码信息"""
        doc = PDFDocument(file_path)
        text_content = []
        pages_info = []

        for page_num, page in enumerate(doc):
            text = page.get_text()
            text_content.append(text)
            pages_info.append({
                "page": page_num + 1,
```

```

        "text": text,
        "blocks": self._extract_blocks(page)
    })

    doc.close()
    return {
        "text": "\n".join(text_content),
        "page_count": len(doc),
        "pages": pages_info,
        "anchors": self._build_page_anchors(pages_info)
    }

async def _parse_image(self, file_path: str) -> Dict[str, Any]:
    """图片 OCR: 提取文字 + 坐标"""
    result = self.ocr.ocr(file_path, cls=True)
    text_lines = []
    anchors = []

    if result and result[0]:
        for line in result[0]:
            bbox, (text, confidence) = line
            text_lines.append(text)
            anchors.append({
                "type": "coordinate",
                "bbox": bbox,
                "text": text,
                "confidence": confidence
            })

    return {
        "text": "\n".join(text_lines),
        "anchors": anchors
    }

async def _parse_audio(self, file_path: str) -> Dict[str, Any]:
    """语音转文字: 带时间戳"""
    result = self.asr.generate(input=file_path)
    transcripts = []
    anchors = []

    for item in result:
        transcripts.append(item["text"])
        anchors.append({
            "type": "timestamp",
            "start": item.get("start", 0),
            "end": item.get("end", 0),
            "text": item["text"]
        })

```

```

    })

    return {
        "text": " ".join(transcripts),
        "duration": self._get_audio_duration(file_path),
        "anchors": anchors
    }

    async def _parse_office(self, file_path: str) -> Dict[str, Any]:
        """Office 文档: 使用 unified 接口"""
        elements = partition(filename=file_path)
        text = "\n\n".join([str(el) for el in elements])
        return {
            "text": text,
            "element_count": len(elements)
        }

```

6.2 双向锚定实现

数据结构:

```

# models/anchor.py
from enum import Enum
from typing import Optional, Dict, Any

class AnchorType(str, Enum):
    PAGE = "page"
    PARAGRAPH = "paragraph"
    TIMESTAMP = "timestamp"
    COORDINATE = "coordinate"

class Anchor:
    def __init__(
        self,
        id: str,
        node_id: str,
        evidence_id: str,
        anchor_type: AnchorType,
        anchor_data: Dict[str, Any]
    ):
        self.id = id
        self.node_id = node_id
        self.evidence_id = evidence_id
        self.anchor_type = anchor_type

```

```

        self.anchor_data = anchor_data

    def to_jump_target(self) -> Dict[str, Any]:
        """转换为前端跳转目标"""
        if self.anchor_type == AnchorType.PAGE:
            return {
                "type": "pdf_page",
                "page": self.anchor_data["page_num"]
            }
        elif self.anchor_type == AnchorType.TIMESTAMP:
            return {
                "type": "audio_time",
                "seconds": self.anchor_data["seconds"]
            }
        elif self.anchor_type == AnchorType.PARAGRAPH:
            return {
                "type": "text_highlight",
                "start": self.anchor_data["start"],
                "end": self.anchor_data["end"]
            }
        elif self.anchor_type == AnchorType.COORDINATE:
            return {
                "type": "image_region",
                "bbox": self.anchor_data["bbox"]
            }

```

跳转逻辑:

```

# services/anchor_service.py
async def get_jump_target(anchor_id: str) -> Dict[str, Any]:
    """获取跳转目标"""
    anchor = await db.get_anchor(anchor_id)
    return anchor.to_jump_target()

```

6.3 时间轴画布实现 (React)

```

// components/Timeline/TimelineCanvas.tsx
import { Timeline } from 'vis-timeline/standalone';
import { useQuery, useMutation } from '@tanstack/react-query';
import { NodeItem } from './types';

interface TimelineCanvasProps {
    caseId: string;

```



```

}

export function TimelineCanvas({ caseId }: TimelineCanvasProps) {
  const { data: nodes, isLoading } = useQuery({
    queryKey: ['nodes', caseId],
    queryFn: () => fetchNodes(caseId),
  });

  const reorderMutation = useMutation({
    mutationFn: (updates: { nodeId: string; sortOrder: number }[]) =>
      fetch('/api/v1/nodes/reorder', {
        method: 'POST',
        body: JSON.stringify({ updates }),
      }),
  });

  const handleDragEnd = (event: any) => {
    const { item, order } = event;
    reorderMutation.mutate([
      { nodeId: item.id, sortOrder: order }
    ]);
  };

  if (isLoading) return <div>Loading...</div>;

  return (
    <div className="timeline-container">
      <Timeline
        items={nodes}
        options={{
          stack: false,
          orientation: 'top',
          editable: true,
          dragTime: true,
        }}
        onMove={handleDragEnd}
      />
      <EvidencePanel nodeId={selectedNodeId} />
    </div>
  );
}

```

6.4 操作日志记录

```
# services/audit_log.py
from functools import wraps
from contextvars import ContextVar

# 上下文变量：当前用户和案件
current_user = ContextVar("current_user", default=None)
current_case = ContextVar("current_case", default=None)

def log_operation(action: str, target_type: str):
    """操作日志装饰器"""
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            # 执行前记录旧值
            old_value = await get_target_value(target_type, kwargs.get('id'))

            # 执行函数
            result = await func(*args, **kwargs)

            # 执行后记录新值
            new_value = await get_target_value(target_type, kwargs.get('id'))

            # 写入日志
            await db.insert_operation_log({
                "user_id": current_user.get(),
                "case_id": current_case.get(),
                "action": action,
                "target_type": target_type,
                "target_id": kwargs.get('id'),
                "old_value": old_value,
                "new_value": new_value,
            })

            return result
        return wrapper
    return decorator

# 使用示例
@log_operation("UPDATE_NODE", "event_node")
async def update_node(node_id: str, updates: dict):
    return await db.update_node(node_id, updates)
```

七、文件存储方案

7.1 存储结构

```
storage/
├── files/                # 原始文件存储
│   ├── {case_id}/
│   │   ├── {uuid}.pdf
│   │   ├── {uuid}.mp3
│   │   └── ...
│   └── parsed/          # 解析结果缓存
│       ├── {case_id}/
│       │   ├── {file_id}.json
│       │   └── ...
└── thumbnails/          # 缩略图
    ├── {case_id}/
    └── {file_id}.jpg
```

7.2 文件上传接口 (FastAPI)

```
# routers/files.py
from fastapi import APIRouter, UploadFile, File, Depends
from fastapi.responses import FileResponse
import aiofiles
import uuid

router = APIRouter(prefix="/files", tags=["files"])

@router.post("/upload")
async def upload_file(
    case_id: str,
    file: UploadFile = File(...),
    current_user: dict = Depends(get_current_user),
):
    # 生成唯一文件名
    file_ext = Path(file.filename).suffix
    stored_name = f"{uuid.uuid4()}{file_ext}"
    stored_path = f"storage/files/{case_id}/{stored_name}"

    # 确保目录存在
    Path(stored_path).parent.mkdir(parents=True, exist_ok=True)
```

```

# 保存文件
async with aiofiles.open(stored_path, 'wb') as f:
    content = await file.read()
    await f.write(content)

# 解析文件
parser = FileParser()
parsed = await parser.parse_file(stored_path, file.content_type)

# 存入数据库
evidence = await db.insert_evidence({
    "case_id": case_id,
    "original_name": file.filename,
    "stored_name": stored_name,
    "file_type": file.content_type,
    "file_size": len(content),
    "parsed_text": parsed["text"],
    **parsed.get("metadata", {})
})

return {"id": evidence.id, "message": "上传成功"}

@router.get("/{file_id}/download")
async def download_file(file_id: str):
    file = await db.get_evidence(file_id)
    file_path = f"storage/files/{file.case_id}/{file.stored_name}"
    return FileResponse(file_path, filename=file.original_name)

```

7.3 大文件处理

- **分块上传**：>10MB 文件启用分块上传
- **断点续传**：记录已上传分块，支持断点续传
- **进度反馈**：WebSocket 实时推送上传进度

八、安全设计

8.1 认证与授权

JWT Token 生成：

```
# services/auth.py
from jose import JWTError, jwt
from datetime import datetime, timedelta

SECRET_KEY = os.getenv("JWT_SECRET")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 # 24 小时

def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def verify_token(token: str) -> dict:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        return None
```

依赖注入验证:

```
async def get_current_user(authorization: str = Header(...)) -> dict:
    token = authorization.replace("Bearer ", "")
    payload = verify_token(token)
    if not payload:
        raise HTTPException(401, "Invalid token")
    return payload
```

8.2 数据隔离

- **案件级隔离**: 每次查询自动添加 `WHERE case_id = ?`
- **权限验证**: 访问文件/节点前验证用户是否有案件访问权
- **SQL 注入防护**: 使用参数化查询

8.3 文件安全

- **存储加密**: 敏感文件使用 Fernet 加密
- **访问控制**: 下载链接需验证权限

- **防外链**：临时下载链接带签名，有效期 1 小时

九、性能优化

9.1 数据库优化

```
# 启用 WAL 模式
await db.execute("PRAGMA journal_mode = WAL")

# 设置缓存大小
await db.execute("PRAGMA cache_size = -64000")

# 异步写入
await db.execute("PRAGMA synchronous = NORMAL")
```

9.2 缓存策略

数据类型	缓存策略	TTL
用户信息	内存缓存 (lru_cache)	30 分钟
案件列表	内存缓存	5 分钟
节点列表	内存缓存	1 分钟
文件预览	磁盘缓存	24 小时
解析结果	磁盘缓存	永久

9.3 懒加载

- **节点列表**：首次加载 50 个，滚动加载更多
- **文件预览**：按需加载，不一次性加载全部
- **全文搜索**：分页返回，限制单次结果数

十、开发计划

10.1 里程碑划分

阶段	周期	交付内容	演示重点
M1	第 1 周	项目骨架、认证、文件上传解析	上传 PDF 可解析文字
M2	第 2 周	节点 CRUD、时间轴、基础排序	创建节点、拖拽排序
M3	第 3 周	双向锚定、证据预览、跳转	点击节点跳转原文
M4	第 4 周	日志、搜索、Tauri 打包	完整流程 + 安装包

10.2 每周迭代计划

第 1 周：基础框架 + 文件解析

- ☐ 项目初始化 (FastAPI + React + Vite)
- ☐ 用户登录/注册 (JWT)
- ☐ 案件 CRUD
- ☐ 文件上传接口
- ☐ PyMuPDF 集成 (PDF 解析)
- ☐ PaddleOCR 集成 (图片 OCR)
- ☐ FunASR 集成 (音频转文字)

第 2 周：节点管理 + 时间轴

- ☐ 节点 CRUD 接口
- ☐ 时间轴前端组件 (vis-timeline)
- ☐ 拖拽排序功能
- ☐ 节点筛选/搜索

- ☐ SQLite 索引优化

第 3 周：双向锚定 + 证据预览

- ☐ 锚定数据结构设计
- ☐ 节点 - 证据关联接口
- ☐ PDF 预览组件 (react-pdf + 页码跳转)
- ☐ 音频播放器 (wavesurfer + 时间戳跳转)
- ☐ 文本高亮定位

第 4 周：完善 + 测试 + 发布

- ☐ 操作日志记录
- ☐ SQLite FTS5 全文搜索
- ☐ Tauri 打包配置
- ☐ 单元测试 + 集成测试
- ☐ 试用安装包打包
- ☐ 部署文档编写

十一、部署方案

11.1 Docker 部署 (推荐)

```
FROM python:3.11-slim

WORKDIR /app

# 安装系统依赖
RUN apt-get update && apt-get install -y \
    libgl1 \
    libglib2.0-0 \
    curl \
    && rm -rf /var/lib/apt/lists/*

# 安装 Python 依赖
```



```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 复制代码
COPY . .

# 创建数据目录
RUN mkdir -p storage data

# 环境变量
ENV PYTHONUNBUFFERED=1
ENV DATABASE_URL=sqlite:///data/legal_minds.db

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

11.2 docker-compose.yml

```
version: '3.8'

services:
  app:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - ./storage:/app/storage
      - ./data:/app/data
    environment:
      - JWT_SECRET=your-secret-key
      - DATABASE_URL=sqlite:///data/legal_minds.db
    restart: unless-stopped

# 前端开发服务器（可选）
frontend:
  image: node:20
  working_dir: /app
  volumes:
    - ./frontend:/app
  command: npm run dev
  ports:
    - "5173:5173"
```

11.3 环境变量配置 (.env)

```
# 服务器配置
SERVER_HOST=0.0.0.0
SERVER_PORT=8000

# 数据库
DATABASE_URL=sqlite:///data/legal_minds.db

# 认证
JWT_SECRET=your-secret-key-here
ACCESS_TOKEN_EXPIRE_MINUTES=1440

# 文件存储
STORAGE_PATH=./storage
MAX_FILE_SIZE=104857600

# 日志
LOG_LEVEL=info

# 功能开关
ENABLE_OCR=true
ENABLE_ASR=true
```

11.4 生产环境检查清单

- ☐ 启用 HTTPS (反向代理配置 SSL)
- ☐ 配置防火墙
- ☐ 设置数据库备份 (定时备份 SQLite 文件)
- ☐ 配置日志轮转 (logrotate)
- ☐ 设置监控告警
- ☐ 压力测试通过 (wrk)

十二、测试策略

12.1 测试类型

类型	工具	覆盖率目标
单元测试	pytest	>80%
集成测试	pytest + httpx	核心流程 100%
E2E 测试	Playwright	关键路径覆盖
性能测试	wrk / locust	QPS>500

12.2 测试用例示例

```
# tests/test_nodes.py
import pytest
from httpx import AsyncClient

@pytest.mark.asyncio
async def test_create_node(client: AsyncClient, auth_headers: dict):
    """测试创建节点"""
    response = await client.post(
        "/api/v1/nodes",
        json={
            "case_id": "case_1",
            "title": "测试事件",
            "event_time": "2024-01-01",
            "description": "测试描述"
        },
        headers=auth_headers
    )
    assert response.status_code == 200
    data = response.json()
    assert data["title"] == "测试事件"
    assert "id" in data

@pytest.mark.asyncio
async def test_update_node(client: AsyncClient, auth_headers: dict):
    """测试更新节点"""
```

```
# 先创建
create_resp = await client.post("/api/v1/nodes", json={...})
node_id = create_resp.json()["id"]

# 更新
response = await client.put(
    f"/api/v1/nodes/{node_id}",
    json={"title": "更新后的标题"},
    headers=auth_headers
)
assert response.status_code == 200
assert response.json()["title"] == "更新后的标题"
```

十三、风险与应对

风险	可能性	影响	应对措施
PyMuPDF 解析失败	低	中	备用方案：pdfplumber
FunASR 识别慢	中	低	异步处理，进度反馈
大文件内存溢出	低	高	流式处理，限制大小
Tauri 打包问题	中	中	早期多端测试
AI 代码质量问题	中	低	代码审查 + 自动化测试
PaddleOCR 模型大	高	低	使用轻量模型 mobile_v3

十四、后续扩展

14.1 二期扩展点

- **AI 自动提取**：集成 LegalOne-R1，自动识别日期/金额/人物

- **多人协作**：WebSocket 实时同步，操作锁机制
- **版本控制**：节点历史版本存储与对比
- **输出层**：证据清单 Word/Excel 模板引擎
- **数据库迁移**：SQLite → PostgreSQL

14.2 技术债务管理

- 代码注释覆盖率>60%
- 每迭代结束重构一次
- 技术文档持续更新
- 定期依赖升级

十五、总结

15.1 技术选型原则

1. **务实优先**：选择成熟稳定的技术，不盲目追求新技术
2. **开发效率**：Python + React 生态丰富，AI 辅助代码质量高
3. **性能够用**：SQLite 首期为简化部署，后期可迁移 PostgreSQL
4. **跨平台**：Tauri 打包体积小，性能优于 Electron

15.2 与商务报价的对应

报价单功能	技术实现
文件上传解析	FastAPI + PyMuPDF + PaddleOCR + FunASR
时间轴画布	React + vis-timeline
双向锚定	自定义 Anchor 数据结构 + 前端跳转
操作日志	装饰器自动记录

报价单功能	技术实现
试用安装包	Tauri 打包 + 功能限制

文档版本 : v1.1

最后更新: 2026 年 3 月 14 日

开发负责人: 王舟

邮箱 : main@mails.wedevs.org

内部技术文档

开发负责人: 王舟 | 电话: 15378391447 | 邮箱: main@mails.wedevs.org